

6. Request flow walk-through

6.1 Sign up

```
Browser —POST /auth/register {name,email,password}→ AuthController
AuthController —register(...)→ AuthService
AuthService —save(user)→ UserRepository → users table
                                   ← ApiResponse{success,message} —
```

6.2 Create an interview & get questions

```
1. POST /interview/create {role, experienceLevel, difficulty, duration}
   → InterviewService.create(...)
     → interviewRepository.save(new Interview(...))
     → questionService.generateAndSave(interviewId, role, level,
difficulty, count)
       → AIService.generateQuestions(...) ★ AI call
       → questionRepository.saveAll(questions)
   ← returns interviewId

2. GET /questions/{interviewId}
   → questionRepository.findByInterviewId(interviewId)
   ← returns List<Question>
```

6.3 Submit & evaluate answers

```
POST /evaluate-answer { answers: [ {questionId, questionText,
answerText}, ... ] }
|
▼
EvaluationController
| 1. answerService.saveAll(answers) → answers table
| 2. evaluationService.evaluateAnswerPairs(pairs)
|   For each (Q, A):
|     AIService.evaluateWithAI(Q, A) ★ AI call
|     if AI fails → fallback rule-based scoring
|     build Evaluation entity → evaluationRepository.save(...)
|     Aggregate: average score, all strengths/weaknesses, etc.
|
▼
Returns EvaluateAnswersResponse {score, fillerWords, confidence,
```

```
relevance, strengths, weaknesses,  
recommendations}
```

6.4 Dashboard

```
GET /api/dashboard → DashboardService.buildDashboard()  
- totalInterviews, averageScore  
- recent interviews (joined with their final score)  
- charts data (scores over time, by role, ...)
```

Module 1 — Auth (AuthController)

1.1 Register a new user

- **Endpoint:** POST /auth/register
- **Service called:** AuthService.register(name, email, password)
- **DB action:** INSERT INTO users (name, email, password) VALUES (...)

cURL

```
curl -X POST http://localhost:8038/auth/register  
-H "Content-Type: application/json"  
-d '{  
  "name": "Bhavya",  
  "email": "bhavya@test.com",  
  "password": "secret123"  
}'
```

Request body (RegisterRequest)

```
{  
  "name": "Bhavya",  
  "email": "bhavya@test.com",  
  "password": "secret123"  
}
```

Success response (ApiResponse) — HTTP 200

```
{ "success": true, "message": "User registered successfully" }
```

Failure response

```
{ "success": false, "message": "Email already registered" }
```

1.2 Login

- **Endpoint:** POST /auth/login
- **Service called:** AuthService.login(email, password)
- **DB action:** SELECT * FROM users WHERE email = ?

cURL

```
curl -X POST http://localhost:8038/auth/login  
-H "Content-Type: application/json"  
-d '{ "email": "bhavya@test.com", "password": "secret123" }'
```

Request (LoginRequest)

```
{ "email": "bhavya@test.com", "password": "secret123" }
```

Success response

```
{ "success": true, "message": "Login successful" }
```

Failure response

```
{ "success": false, "message": "Invalid email or password" }
```

Module 2 — Interview lifecycle

(Interviewcontroller)

2.1 Create a new interview

- **Endpoint:** POST /interview/create
- **Service called:** InterviewService.create(...)
- **DB actions:**

- `INSERT INTO interviews (...)`
- `INSERT INTO questions (...)`

cURL

```
curl -X POST http://localhost:8038/interview/create
-H "Content-Type: application/json"
-d '{
  "role": "Java Developer",
  "experienceLevel": "Junior",
  "difficulty": "Medium",
  "duration": 15
}'
```

Request (InterviewRequest)

```
{
  "role": "Java Developer",
  "experienceLevel": "Junior",
  "difficulty": "Medium",
  "duration": 15
}
```

Response (InterviewResponse)

```
{
  "success": true,
  "message": "Interview created",
  "interviewId": 42
}
```

2.2 Get the questions for an interview

- **Endpoint:** `GET /questions/{interviewId}`
- **Service called:** `QuestionService.getQuestionsForInterview(interviewId)`

cURL

```
curl http://localhost:8038/questions/42
```

Response

```
[
  { "id": 101, "interviewId": 42, "questionText": "What is the difference
between an ArrayList and a LinkedList in Java?" },
  { "id": 102, "interviewId": 42, "questionText": "Explain how the JVM manages
memory between heap and stack." }
]
```

2.3 Finish an interview

- **Endpoint:** `POST /interview/{id}/finish`
- **Service called:** `InterviewService.finish(id, finalScore)`

cURL

```
curl -X POST http://localhost:8038/interview/42/finish
-H "Content-Type: application/json"
-d '{ "finalScore": 78 }'
```

Response

```
{ "success": true, "message": "Interview finished", "interviewId": 42 }
```

Module 3 — Answers (AnswerController)

3.1 Save the candidate's answers

- **Endpoint:** `POST /answers`
- **Service called:** `AnswerService.saveAll(answers)`

cURL

```
curl -X POST http://localhost:8038/answers
-H "Content-Type: application/json"
-d '{
  "answers": [
    { "questionId": 101, "answerText": "ArrayList uses a dynamic array." }
  ]
}'
```

Response

```
{ "success": true, "saved": 2 }
```

Module 4 — Evaluation (EvaluationController)

★ AI flow

4.1 Submit answers AND evaluate them with AI

- **Endpoint:** POST /evaluate-answer
- **Services called:**
 - AnswerService.saveAll(...)
 - EvaluationService.evaluateAnswerPairs(...)
 - AIService.evaluateWithAI(...)

cURL

```
curl -X POST http://localhost:8038/evaluate-answer
-H "Content-Type: application/json"
-d '{
  "answers": [
    {
      "questionId": 101,
      "questionText": "What is the difference between ArrayList and
LinkedList?",
      "answerText": "ArrayList uses a dynamic array ..."
    }
  ]
}'
```

Response (EvaluateAnswersResponse)

```
{
  "success": true,
  "score": 74,
  "fillerWords": 3,
  "confidence": 82,
  "relevances": "high"
}
```

Module 5 — Dashboard (DashboardController)

5.1 Get the dashboard data

- **Endpoint:** GET /api/dashboard
- **Service called:** DashboardService.buildDashboard()

cURL

```
curl http://localhost:8038/api/dashboard
```

Response (DashboardResponse)

```
{
  "totalInterviews": 12,
  "averageScore": 71.5,
  "bestScore": 92
}
```

Module 6 — Health check (TestController)

6.1 Verify backend is up

- **Endpoint:** GET /test

cURL

```
curl http://localhost:8038/test
```

Response

```
Backend is working 🚀
```

Endpoint cheat-sheet

#	Method	Endpoint	Module
1	POST	/auth/register	Auth
2	POST	/auth/login	Auth
3	POST	/interview/create	Interview
4	GET	/questions/{interviewId}	Interview
5	POST	/interview/{id}/finish	Interview
6	POST	/answers	Answer
7	POST	/evaluate-answer	Evaluation
8	GET	/api/dashboard	Dashboard
9	GET	/test	Health